

Critical, atomic, privatização e reduction no OpenMP

Matheus Marinho

13 de setembro de 2026

1. Objetivo

O experimento estima π pelo método de Monte Carlo. Para cada um dos N pontos, duas coordenadas pseudoaleatórias são geradas no quadrado unitário; se $x^2 + y^2 \leq 1$, ocorre um *hit*. Ao final:

$$\pi \approx 4 \times \text{hits} / N$$

O estudo compara cinco maneiras corretas de acumular os hits em paralelo:

Versão	Contagem durante o laço	Combinação global explícita
compartilhado + <code>critical</code>	contador global	uma por hit
compartilhado + <code>atomic</code>	contador global	uma por hit
privado + <code>critical</code>	contador de cada thread	uma por thread
privado + <code>atomic</code>	contador de cada thread	uma por thread
<code>reduction</code>	cópia privada administrada pelo OpenMP	nenhuma no código

Todas as versões usam `rand_r`, sementes privadas por thread e `schedule(static)`. Com a mesma semente e equipe, cada thread gera a mesma sequência nas cinco versões. Assim, diferenças de resultado indicariam erro de sincronização, enquanto diferenças de tempo refletem a estratégia de acumulação.

2. Por que `rand_r` e estado privado

`rand()` mantém estado global oculto. Chamá-lo concorrentemente pode exigir sincronização interna ou causar uma corrida, conforme a implementação. `rand_r` recebe o endereço do estado que deve atualizar. Cada thread mantém sua variável `estado` na pilha:

```
int id = omp_get_thread_num();
unsigned int estado = semente_da_thread(mente, id);
double x = (double)rand_r(&estado) / (double)RAND_MAX;
```

Portanto, threads diferentes não escrevem no mesmo estado do gerador. No Windows, onde a UCRT não oferece a extensão POSIX `rand_r`, o fonte inclui uma implementação compatível para permitir o mesmo experimento. O objetivo não é avaliar a qualidade estatística do gerador, mas isolar os mecanismos OpenMP.

3. Contador compartilhado com `critical`

Na primeira versão, cada hit entra em uma região crítica e incrementa o contador:

```
if (ponto_dentro_do_circulo(&estado)) {
    #pragma omp critical
    {
        hits_compartilhados++;
    }
}
```

`critical` aceita um bloco arbitrário e garante que apenas uma thread por vez execute uma região com o mesmo nome. É uma ferramenta geral: poderia proteger várias instruções, campos relacionados ou uma operação sobre uma estrutura. Aqui, porém, o bloco possui apenas um incremento e é executado em aproximadamente 78,5% das iterações. Milhões de aquisições serializam a parte mais frequente do laço e produzem grande contenção.

4. Contador compartilhado com `atomic`

A segunda versão preserva o contador global, mas protege somente a atualização:

```
if (ponto_dentro_do_circulo(&estado)) {
    #pragma omp atomic update
```

```
    hits_compartilhados++;  
}
```

`atomic` não protege um bloco geral. Ele se aplica a formas de acesso reconhecidas pelo OpenMP, como `x++`, `x += expressao` e operações de leitura, escrita, captura ou comparação admitidas pela versão da especificação. O compilador normalmente pode gerar uma operação atômica de hardware ou uma sequência especializada, sem o protocolo mais geral de entrada e saída de uma região crítica. Por isso tende a ser mais barato que `critical` para um único contador.

Atômico não significa paralelo sobre a mesma posição: as atualizações ainda precisam ser ordenadas pelo sistema de coerência de cache. Logo, o contador continua sendo um ponto de contenção e o custo cresce com a frequência de hits e o número de threads.

5. Privatização seguida de combinação

As versões seguintes removem o contador global do caminho quente. Cada thread incrementa uma variável local, que normalmente permanece em registrador, e só combina seu subtotal depois do `omp for`:

```
long long hits_privados = 0;  
  
#pragma omp for schedule(static)  
for (long long i = 0; i < pontos; i++) {  
    hits_privados += ponto_dentro_do_circulo(&estado);  
}  
  
#pragma omp atomic update  
hits_compartilhados += hits_privados;
```

Há uma versão com `critical` e outra com `atomic` na soma final. Com `T` threads, cada thread executa uma atualização global, totalizando `T`, em vez de aproximadamente $0,785 \times N$. No teste, isso reduz cerca de 15,7 milhões de sincronizações para apenas oito. Como a frequência domina o custo, a escolha entre `critical` e `atomic` na combinação final quase desaparece nas medições.

Essa transformação exige que a operação admita subtotais independentes e uma combinação associativa. Também acrescenta código manual, criando oportunidades para esquecer a inicialização, a combinação ou a proteção final.

6. Versão com reduction

`reduction(+:hits)` expressa diretamente o mesmo padrão de privatizar e combinar:

```
long long hits = 0;  
  
#pragma omp parallel default(none) shared(pontos, semente) \  
    reduction(+:hits)  
{  
    /* estado de rand_r privado por thread */  
    #pragma omp for schedule(static)  
    for (long long i = 0; i < pontos; i++) {  
        hits += ponto_dentro_do_circulo(&estado);  
    }  
}
```

O runtime cria uma cópia privada de `hits` para cada thread, inicializa-a com o elemento neutro da soma e combina as cópias ao final. A tabela do programa mostra zero atualizações **explicitamente** sincronizadas porque não há `critical` nem `atomic` no fonte; ainda existe uma etapa interna de redução.

Quando a operação e o tipo são suportados, `reduction` oferece simultaneamente boa eficiência e alta produtividade: o código é menor, documenta a intenção e delega ao compilador/runtime a estratégia de combinação.

7. Metodologia e validação

O programa foi compilado sem avisos com:

```
gcc -O2 -Wall -Wextra -std=c11 -fopenmp pi_sincronizacao.c -o pi_sincronizacao.exe
```

Antes da medição, uma região paralela de aquecimento cria a equipe. Cada versão processa 20 milhões de pontos com oito threads. Foram realizadas cinco repetições com sementes consecutivas; o tempo é de parede, medido por `omp_get_wtime`.

Em cada repetição, as cinco versões produziram exatamente o mesmo número de hits e o programa imprimiu `Validacao: OK`. Os valores de π variaram aproximadamente entre 3,14150 e 3,14173, comportamento esperado de uma estimativa estocástica.

8. Resultados

Versão	Atualizações explícitas por execução	Tempos (s)	Mediana (s)	Razão contra <code>critical</code> compartilhado
compartilhado + <code>critical</code>	≈ 15,7 milhões	1,086; 1,122; 1,088; 1,074; 1,098	1,088	1,0×
compartilhado + <code>atomic</code>	≈ 15,7 milhões	0,190; 0,190; 0,191; 0,196; 0,188	0,190	5,7×
privado + <code>critical</code>	8	0,020; 0,018; 0,019; 0,021; 0,018	0,019	57,3×
privado + <code>atomic</code>	8	0,018; 0,019; 0,019; 0,018; 0,020	0,019	57,3×
<code>reduction</code>	0 no fonte	0,018; 0,019; 0,018; 0,019; 0,019	0,019	57,3×

O `atomic` compartilhado foi aproximadamente 5,7 vezes mais rápido que o `critical` compartilhado porque sua operação é mais restrita e pode ser implementada de modo especializado. Entretanto, privatizar foi muito mais importante do que trocar a primitiva: as versões privadas e a redução foram cerca de 57 vezes mais rápidas que a primeira versão.

As diferenças de um milissegundo entre as três melhores versões são pequenas e não estabelecem uma vencedora universal. Nesse tamanho de teste, todas gastam quase todo o tempo gerando e classificando pontos; somente oito combinações finais têm custo desprezível. A redução é preferível aqui por expressar a intenção com menos código, mesmo sem ter a menor mediana isolada.

9. Desempenho, produtividade e roteiro de decisão

Uma estratégia prática é procurar primeiro uma formulação que evite estado compartilhado frequente e somente depois escolher a primitiva:

- A operação é uma soma, produto, mínimo, máximo, operação lógica ou outra redução suportada?** Usar `reduction`. É concisa, difícil de usar incorretamente e normalmente escalável.
- É possível acumular privadamente e combinar uma vez por thread/tarefa?** Privatizar, mesmo quando a combinação final precisar de `atomic` ou `critical`. Reduzir a frequência de sincronização costuma valer mais que trocar de primitiva.
- Resta uma atualização simples de uma única posição?** Usar `atomic`. É leve, mas apenas quando a forma aceita pelo OpenMP preserva todo o invariante.
- É necessário proteger várias instruções ou vários campos como uma única operação?** Usar `critical`. É simples e legível, porém serializa o bloco.
- Há poucos recursos independentes conhecidos na compilação?** Usar regiões críticas nomeadas, por exemplo `critical(fila_a)` e `critical(fila_b)`, para que recursos diferentes não se bloqueiem desnecessariamente.
- A quantidade de recursos é dinâmica, é preciso escolher a proteção por índice, tentar adquirir sem bloquear ou controlar o ciclo de vida?** Usar locks explícitos (`omp_init_lock`, `omp_set_lock`, `omp_unset_lock` e `omp_destroy_lock`). Eles oferecem mais controle, mas exigem mais código e cuidado com esquecimento de liberação, ordem de aquisição, deadlock e falso compartilhamento entre locks vizinhos.

Nenhuma ferramenta é sempre mais rápida. A região protegida, a frequência, a contenção e a arquitetura determinam o resultado. Em produtividade, `reduction` é a melhor abstração para este problema; `atomic` é a solução mínima para uma atualização simples; `critical` favorece clareza em invariantes maiores; e locks explícitos ficam reservados para estruturas que realmente precisam do controle dinâmico adicional.

10. Conclusão

As cinco versões são corretas, mas expõem custos muito diferentes. Substituir `critical` por `atomic` reduz o custo de cada incremento, sem eliminar a contenção no contador compartilhado. Privatizar altera a escala do problema: o número de operações sincronizadas deixa de ser proporcional aos hits e passa a ser proporcional às threads. `reduction` realiza essa privatização com a descrição mais curta e direta, sendo a escolha natural para a soma de hits.

11. Código-fonte completo

pi_sincronizacao.c · 278 linhas

```
1 /*
2  * Tarefa 10 - critical, atomic, contadores privados e reduction
3  *
4  * Compilar:
5  * gcc -O2 -Wall -Wextra -std=c11 -fopenmp pi_sincronizacao.c -o pi_sincronizacao.exe
6  *
7  * Executar:
8  * pi_sincronizacao.exe [pontos] [threads] [semente]
9  */
10
11 #define _POSIX_C_SOURCE 199506L
12
13 #include <errno.h>
14 #include <limits.h>
15 #include <omp.h>
16 #include <stdio.h>
17 #include <stdlib.h>
18
19 /* rand_r nao existe na UCRT/MinGW. No Windows, esta funcao compativel permite
20  * executar o estudo; no Linux, e usada a implementacao POSIX da libc. */
21 #ifdef _WIN32
22 static int rand_r(unsigned int *semente)
23 {
24     unsigned int proximo = *semente;
25     unsigned int resultado;
26
27     proximo = proximo * 1103515245u + 12345u;
28     resultado = (proximo / 65536u) % 2048u;
29     proximo = proximo * 1103515245u + 12345u;
30     resultado = (resultado << 10) ^ ((proximo / 65536u) % 1024u);
31     proximo = proximo * 1103515245u + 12345u;
32     resultado = (resultado << 10) ^ ((proximo / 65536u) % 1024u);
33     *semente = proximo;
34     return (int)(resultado % ((unsigned int)RAND_MAX + 1u));
35 }
36 #endif
37
38 typedef struct {
39     const char *nome;
40     long long hits;
41     long long atualizacoes_explicitas;
42     double pi;
43     double segundos;
44 } Resultado;
45
46 static long long hits_compartilhados;
47
48 static unsigned int semente_da_thread(unsigned int base, int id)
49 {
50     return base + 0x9e3779b9u * (unsigned int)(id + 1);
51 }
52
53 static int ponto_dentro_do_circulo(unsigned int *estado)
54 {
55     double x = (double)rand_r(estado) / (double)RAND_MAX;
56     double y = (double)rand_r(estado) / (double)RAND_MAX;
57     return x * x + y * y <= 1.0;
58 }
59
60 static Resultado finalizar(const char *nome, long long hits,
61                           long long sincronizacoes, long long pontos,
62                           double inicio)
63 {
64     Resultado r;
65     r.nome = nome;
66     r.hits = hits;
67     r.atualizacoes_explicitas = sincronizacoes;
68     r.pi = 4.0 * (double)hits / (double)pontos;
69     r.segundos = omp_get_wtime() - inicio;
70     return r;
71 }
72
73 /* 1) Cada hit atualiza imediatamente o contador compartilhado sob critical. */
74 static Resultado compartilhado_critical(long long pontos,
75                                         unsigned int semente)
76 {
77     double inicio = omp_get_wtime();
78     hits_compartilhados = 0;
79
80     #pragma omp parallel default(none) \
81         shared(pontos, semente, hits_compartilhados)
82     {
```

```

83     int id = omp_get_thread_num();
84     unsigned int estado = semente_da_thread(semente, id);
85
86     #pragma omp for schedule(static)
87     for (long long i = 0; i < pontos; i++) {
88         if (ponto_dentro_do_circulo(&estado)) {
89             #pragma omp critical
90             {
91                 hits_compartilhados++;
92             }
93         }
94     }
95 }
96
97 return finalizar("compartilhado + critical", hits_compartilhados,
98                hits_compartilhados, pontos, inicio);
99 }
100
101 /* 2) Cada hit atualiza imediatamente o contador com uma operacao atomica. */
102 static Resultado compartilhado_atomic(long long pontos,
103                                       unsigned int semente)
104 {
105     double inicio = omp_get_wtime();
106     hits_compartilhados = 0;
107
108     #pragma omp parallel default(none) \
109     shared(pontos, semente, hits_compartilhados)
110     {
111         int id = omp_get_thread_num();
112         unsigned int estado = semente_da_thread(semente, id);
113
114         #pragma omp for schedule(static)
115         for (long long i = 0; i < pontos; i++) {
116             if (ponto_dentro_do_circulo(&estado)) {
117                 #pragma omp atomic update
118                 hits_compartilhados++;
119             }
120         }
121     }
122
123     return finalizar("compartilhado + atomic", hits_compartilhados,
124                    hits_compartilhados, pontos, inicio);
125 }
126
127 /* 3) Cada thread conta localmente e entra no critical somente uma vez. */
128 static Resultado privado_critical(long long pontos, unsigned int semente,
129                                   int threads)
130 {
131     double inicio = omp_get_wtime();
132     hits_compartilhados = 0;
133
134     #pragma omp parallel default(none) \
135     shared(pontos, semente, hits_compartilhados)
136     {
137         int id = omp_get_thread_num();
138         unsigned int estado = semente_da_thread(semente, id);
139         long long hits_privados = 0;
140
141         #pragma omp for schedule(static)
142         for (long long i = 0; i < pontos; i++) {
143             hits_privados += ponto_dentro_do_circulo(&estado);
144         }
145
146         #pragma omp critical
147         {
148             hits_compartilhados += hits_privados;
149         }
150     }
151
152     return finalizar("privado + critical", hits_compartilhados,
153                    threads, pontos, inicio);
154 }
155
156 /* 4) Cada thread conta localmente e faz somente uma soma atomica no final. */
157 static Resultado privado_atomic(long long pontos, unsigned int semente,
158                                 int threads)
159 {
160     double inicio = omp_get_wtime();
161     hits_compartilhados = 0;
162
163     #pragma omp parallel default(none) \
164     shared(pontos, semente, hits_compartilhados)
165     {
166         int id = omp_get_thread_num();
167         unsigned int estado = semente_da_thread(semente, id);
168         long long hits_privados = 0;
169
170         #pragma omp for schedule(static)
171         for (long long i = 0; i < pontos; i++) {

```

```

172     hits_privados += ponto_dentro_do_circulo(&estado);
173 }
174
175 #pragma omp atomic update
176 hits_compartilhados += hits_privados;
177 }
178
179 return finalizar("privado + atomic", hits_compartilhados,
180                threads, pontos, inicio);
181 }
182
183 /* 5) O OpenMP cria acumuladores privados e os combina automaticamente. */
184 static Resultado com_reduction(long long pontos, unsigned int semente)
185 {
186     long long hits = 0;
187     double inicio = omp_get_wtime();
188
189     #pragma omp parallel default(none) shared(pontos, semente) \
190         reduction(+:hits)
191     {
192         int id = omp_get_thread_num();
193         unsigned int estado = semente_da_thread(semente, id);
194
195         #pragma omp for schedule(static)
196         for (long long i = 0; i < pontos; i++) {
197             hits += ponto_dentro_do_circulo(&estado);
198         }
199     }
200
201     return finalizar("reduction", hits, 0, pontos, inicio);
202 }
203
204 static int ler_ll(const char *texto, long long minimo, long long *valor)
205 {
206     char *fim;
207     long long convertido;
208
209     errno = 0;
210     convertido = strtoll(texto, &fim, 10);
211     if (errno != 0 || *texto == '\0' || *fim != '\0' || convertido < minimo) {
212         return 0;
213     }
214     *valor = convertido;
215     return 1;
216 }
217
218 static void imprimir(Resultado r)
219 {
220     printf("%-25s %12lld %.9f %12lld %11.6f\n",
221           r.nome, r.hits, r.pi, r.atualizacoes_explicitas, r.segundos);
222 }
223
224 int main(int argc, char **argv)
225 {
226     long long pontos = 20000000;
227     long long threads_lido = 8;
228     long long semente_lida = 2026;
229     Resultado resultados[5];
230
231     if (argc > 4 ||
232         (argc > 1 && !ler_ll(argv[1], 1, &pontos)) ||
233         (argc > 2 && !ler_ll(argv[2], 1, &threads_lido)) ||
234         (argc > 3 && !ler_ll(argv[3], 0, &semente_lida)) ||
235         threads_lido > INT_MAX || semente_lida > UINT_MAX) {
236         fprintf(stderr, "Uso: %s [pontos>=1] [threads>=1] [semente]\n",
237             argv[0]);
238         return EXIT_FAILURE;
239     }
240
241     int threads = (int)threads_lido;
242     unsigned int semente = (unsigned int)semente_lida;
243     int aquecimento = 0;
244
245     omp_set_dynamic(0);
246     omp_set_num_threads(threads);
247
248     /* Cria a equipe antes das medicoes para reduzir o efeito de inicializacao. */
249     #pragma omp parallel reduction(+:aquecimento)
250     aquecimento += omp_get_thread_num() >= 0;
251     if (aquecimento != threads) {
252         threads = aquecimento;
253     }
254
255     resultados[0] = compartilhado_critical(pontos, semente);
256     resultados[1] = compartilhado_atomic(pontos, semente);
257     resultados[2] = privado_critical(pontos, semente, threads);
258     resultados[3] = privado_atomic(pontos, semente, threads);
259     resultados[4] = com_reduction(pontos, semente);
260

```

```
261     printf("Tarefa 10 - sincronizacao da soma de hits com rand_r\n");
262     printf("Pontos: %lld | threads: %d | semente: %u\n\n",
263           pontos, threads, semente);
264     printf("Versao          Hits          pi  Atualiz. explic. Tempo (s)\n");
265     for (int i = 0; i < 5; i++) {
266         imprimir(resultados[i]);
267     }
268
269     int valido = 1;
270     for (int i = 1; i < 5; i++) {
271         if (resultados[i].hits != resultados[0].hits) {
272             valido = 0;
273         }
274     }
275     printf("\nValidacao (mesmo numero de hits): %s\n",
276           valido ? "OK" : "FALHOU");
277     return valido ? EXIT_SUCCESS : EXIT_FAILURE;
278 }
```